Це CS50

компіляція

препроцессінг

компіляція

збирання

компонування

```
#include <cs50.h>
#include <stdio.h>


int main(void)
{
    string name = get_string("Name: ");
    printf("hello, %s\n", name);
}
```

```
...
string get_string(string prompt);
int printf(const char *format, ...);
...

int main(void)
{
    string name = get_string("Name: ");
    printf("hello, %s\n", name);
}
```

```
...
main:                                   # @main
    .cfi_startproc
# BB#0:
    pushq    %rbp
.Ltmp0:
    .cfi_def_cfa_offset 16
.Ltmp1:
    .cfi_offset %rbp, -16
    movq    %rsp, %rbp
.Ltmp2:
    .cfi_def_cfa_register %rbp
    subq    $16, %rsp
    xorl    %eax, %eax
    movl    %eax, %edi
    movabsq    $.L.str, %rsi
    movb    $0, %al
    callq    get_string
    movabsq    $.L.str.1, %rdi
    movq    %rax, -8(%rbp)
    movq    -8(%rbp), %rsi
    movb    $0, %al
    callq    printf
    ...
```

```
01111111010001010100110001000110
00000010000000010000000100000000
00000000000000000000000000000000
00000000000000000000000000000000
00000001000000000001111100000000
00000001000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
10100000000000100000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
01000000000000000000000000000000
00000000000000000100000000000000
00001010000000000000000100000000
01010101010010001000100111100101
01001000100000111110110000010000
00110001110000001000100111000111
01001000101111100000000000000000
00000000000000000000000000000000
00000000000000001011000000000000
11101000000000000000000000000000
00000000010010001011111100000000
00000000000000000000000000000000
00000000000000000000000001001000
...
```

```
01111111010001010100110001000110
00000010000000010000000100000000
00000000000000000000000000000000
00000000000000000000000000000000
00000001000000000011111000000000
00000001000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
10100000000000100000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
01000000000000000000000000000000
00000000000000000100000000000000
00001010000000000000000100000000
01010101010010001000100111100101
01001000100000111110110000010000
00110001110000001000100111000111
01001000101111100000000000000000
00000000000000000000000000000000
00000000000000001011000000000000
11101000000000000000000000000000
00000000010010001011111100000000
00000000000000000000000000000000
00000000000000000000000001001000
...
```

cs50.c

printf.c

```
011111110100010101001100010001100
000000100000000010000000100000000
000000000000000000000000000000000
000000000000000000000000000000000
000000010000000000111110000000000
000000010000000000000000000000000
000000000000000000000000000000000
000000000000000000000000000000000
000000000000000000000000000000000
000000000000000000000000000000000
101000000000001000000000000000000
000000000000000000000000000000000
000000000000000000000000000000000
010000000000000000000000000000000
000000000000000000010000000000000
000010100000000000000000100000000
010101010100100010001001111001 01
010010001000001111101100000100 00
001100011100000010001001110001 11
010010001011111000000000000000000
000000000000000000000000000000000
000000000000000010110000000000000
111010000000000000000000000000000
000000000100100010111111000000000
000000000000000000000000000000000
000000000000000000000000001001000
...
```

```
011111110100010101001100010001100
000000100000000010000000100000000
000000000000000000000000000000000
000000000000000000000000000000000
000000110000000000111110000000000
000000010000000000000000000000000
110000000000111100000000000000000
000000000000000000000000000000000
010000000000000000000000000000000
000000000000000000000000000000000
001010000011001000000000000000000
000000000000000000000000000000000
000000000000000000000000000000000
010000000000000000011100000000000
000001110000000001000000000000000
000111000000000000011001000000000
000000010000000000000000000000000
000001010000000000000000000000000
000000000000000000000000000000000
000000000000000000000000000000000
000000000000000000000000000000000
000000000000000000000000000000000
010111000010010100000000000000000
000000000000000000000000000000000
...
```

```
001011110110110001101001011000 10
011000110010111001110011011011 11
001011100011011000100000001011 11
011101010111001101110010001011 11
011011000110100101100010001011 11
011110000011100000110110010111 11
001101100011010000101101011011 00
011010010110111001110101011110 00
001011010110011101101110011101 01
001011110110110001101001011000 10
011000110101111101101110011011 11
011011100111001101101000011000 01
011100100110010101100100001011 10
011000010010000000100000010000 01
010100110101111101001110010001 01
010001010100010001000101010001 00
001000000010100000100000001011 11
011011000110100101100010001011 11
011110000011100000110110010111 11
001101100011010000101101011011 00
011010010110111001110101011110 00
001011010110011101101110011101 01
001011110110110001100100001011 01
011011000110100101101110011101 01
011110000010110101111000001110 00
001101100010110100110110001101 00
...
```

0111111101000101010011000100011000000100000000100000001000000000000000000000000000000000000000000000
0000000000000000000000000000000100000000001111100000000000000100000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000010100000000000100000000000000000000000000000000000000000000000000000000000000000000000
0000000001000000000000000000000000000000000000000000000100000000000000000101000000000000000001000000
0001010101010010001000100111100101010010001000001111101100000100000011000111000000100010011100011101 00
1000101111100000000000000000000000000000000000000000000000000000000000000000000000000101100000000000111010000 0
0000000000000000000000000000001001000101111110000000000000000000000000000000000000000000000000000000
0000000001001000...0111111101000101010011000100011000000100000000100000001000000000000000000000000000
0000000000000000000000000000000000000000000000000011000000000011111000000000000000001000000000000000
0000000110000000000111100000000000000000000000000000000000000000000000000000000100000000000000000000
0000000000000000000000000000000000001010000011001000000000000000000000000000000000000000000000000000
0000000000000000000000000000001000000000000000011100000000000000011100000000010000000000000000011100000
0000000011001000000000000000100000000000000000000000000000101000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000000000000000000000101110000100101000000000000
0000000000000000000000000000000000000000...0010111110110110001101001011000100110001100101110011100110110111 1
0010111100011011000100000001011110111010101110011011100100010111101101100011010010110001000101111011110
0000111000001101100101111100110110001101000010110101101100011010010110111001110101011110000010110101 10
0111011011100111010100101111011011000110100101100010011000110101111101101110011011110110111001110011 01
1010000110000101110010011001010110010000101110011000010010000000100000010000010101001101011111010011 10
0100010101000101010001000100010101000100001000000010100000100000001011110110110001101001011000100010 11
1101111000001110000011011001011111001101100011010000101101011011000110100101101110011101010111100000 10
1101011001110110111001110101001011110110110001100100001011010110110001101001011011100111010101111000 00
1011010111100000111000001101100010110100110110001101 00...

help50

printf

style50

CS50 IDE

CS50 IDE

CS50 IDE File Edit Find View Go Share

~/workspace/

hello.c

hello.c

```
#include <stdio.h>

int main(void)
{
    printf("hello, world\n");
}
```

workspace/

```
~/workspace/ $
```

Collaborate Outline Debugger

CS50 IDE

CS50 IDE File Edit Find View Go Share

~/workspace/

hello.c

hello.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("hello, world\n");
6 }
```

workspace/

```
~/workspace/ $
```

Collaborate Outline Debugger

cd

ls

mkdir

rm

rmdir

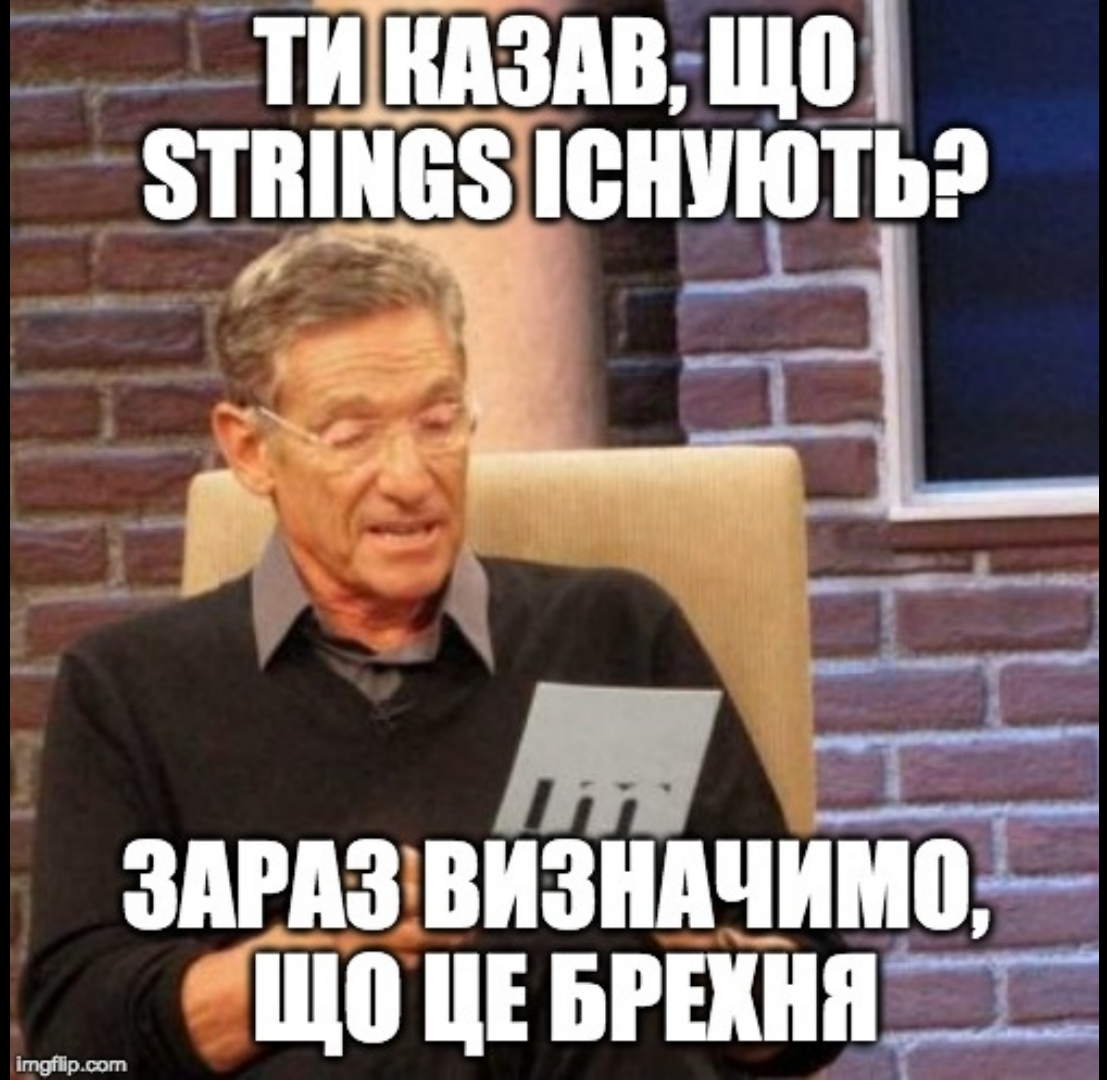
…

check50

debug50

```
get_char

get_double

get_float

get_int

get_long

get_string

...
```

string


ТИ КАЗАВ, ЩО
STRINGS ІСНУЮТЬ?
ЗАРАЗ ВИЗНАЧИМО,
ЩО ЦЕ БРЕХНЯ
imgflip.com

string

char *

malloc

free

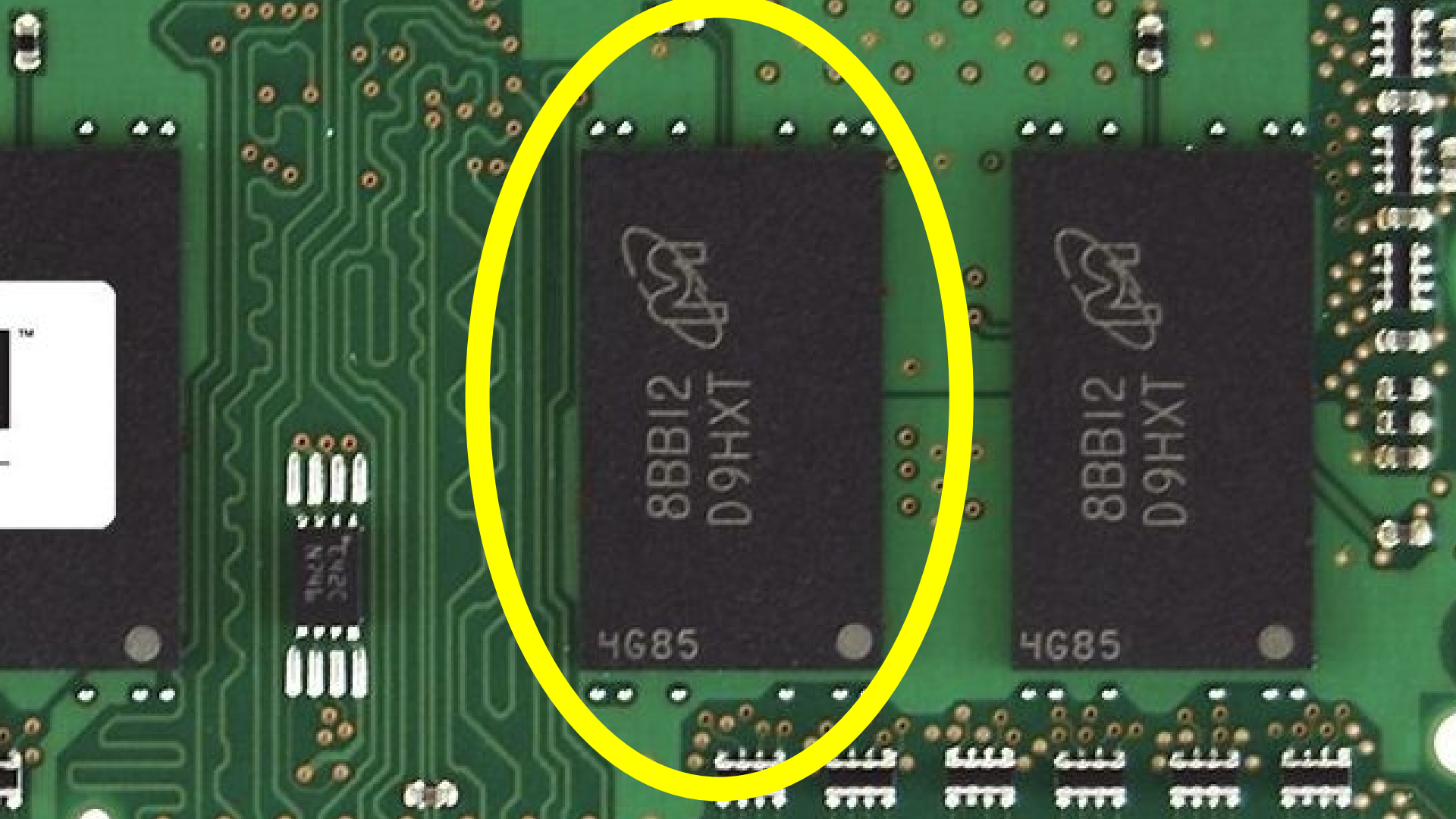
…

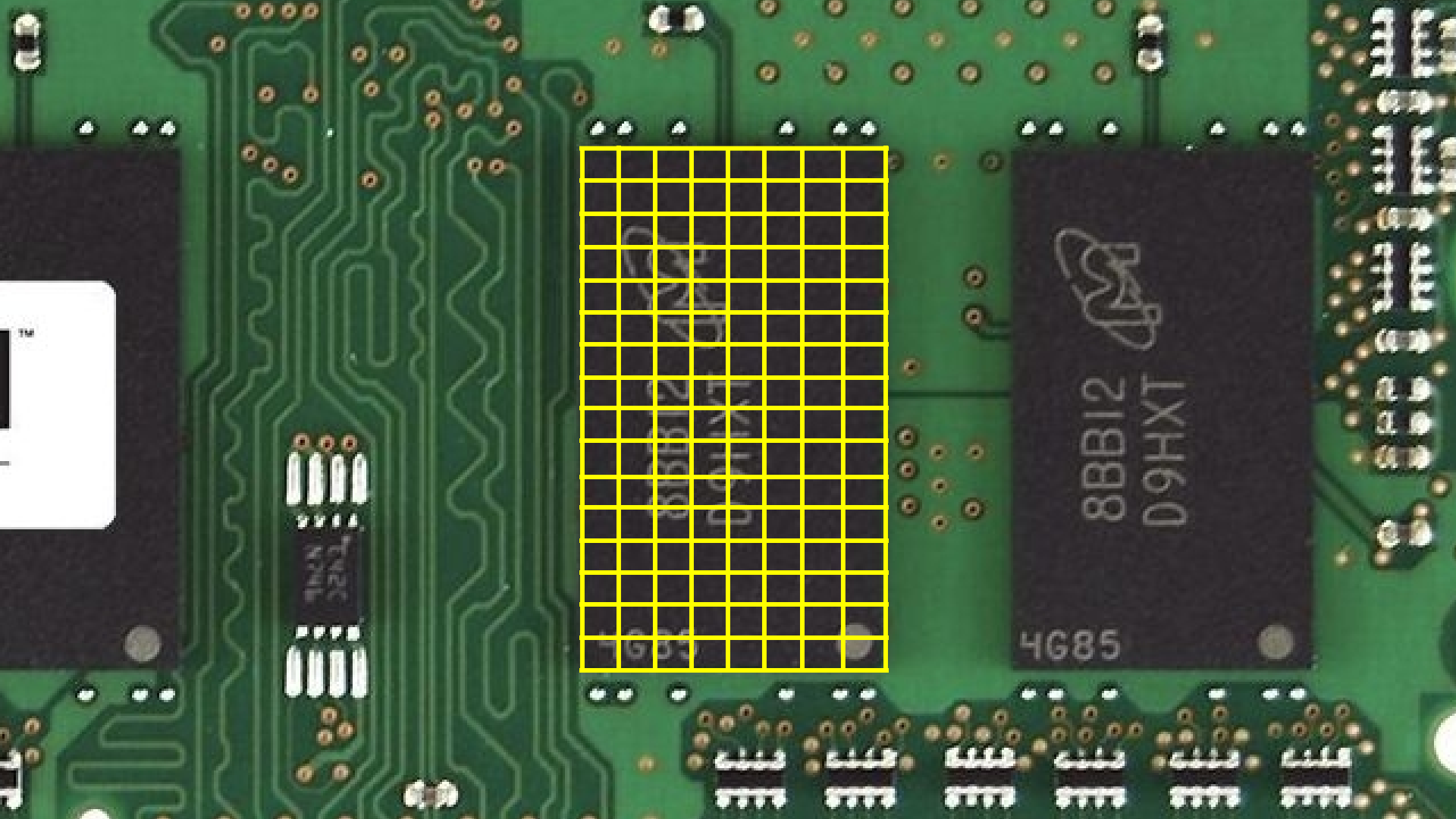
crucial
by Micron
47J6
4G85
8BB12
D9HXT
4G85
8BB12
D9HXT
4G85

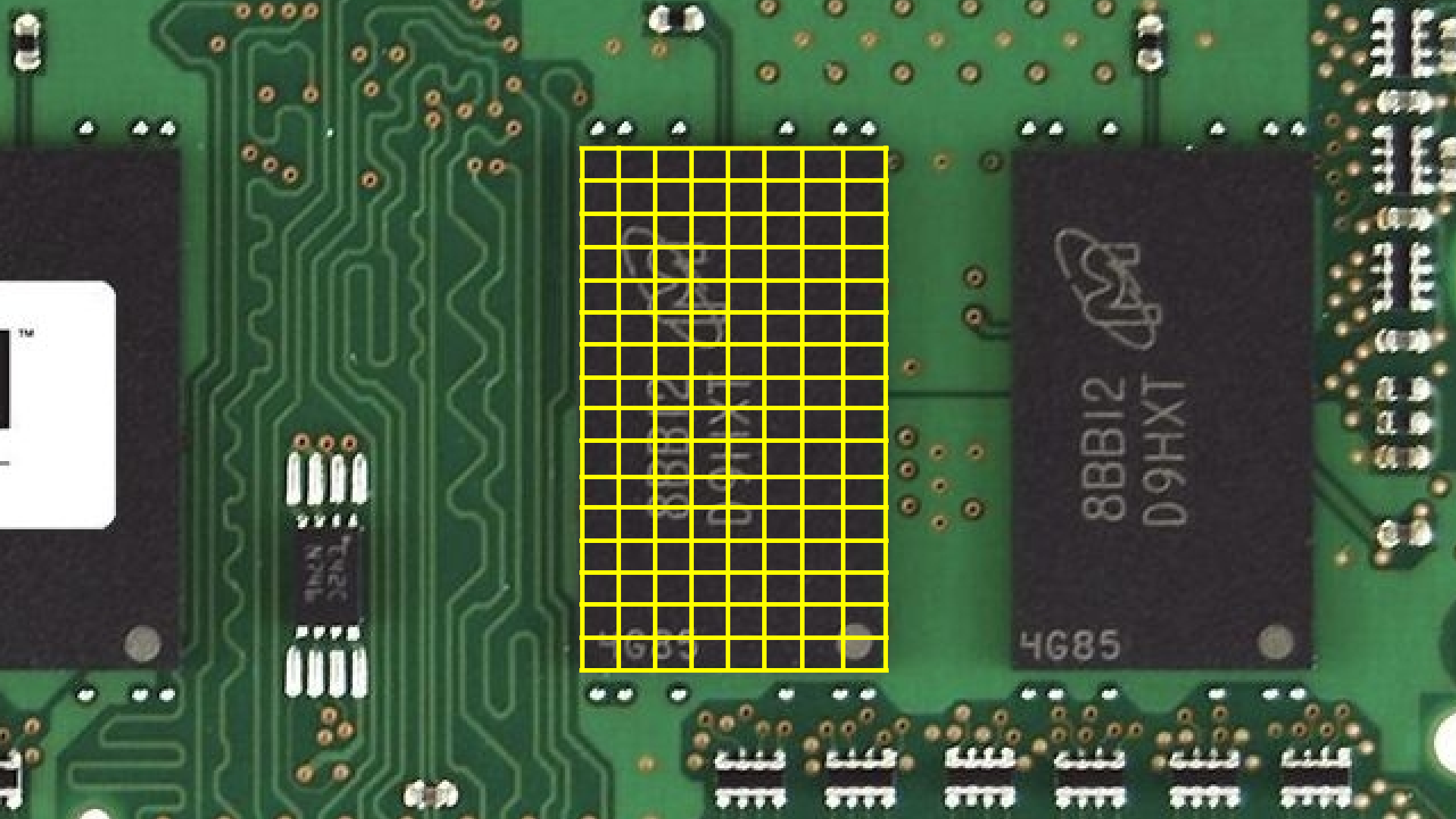
amazon.com

crucial
by Micron
8BB12
D9HXT
8BB12
D9HXT
47J6
4G85
4G85
4G85

amazon.com


8BBI2
D9HXT
4G85
8BBI2
D9HXT
4G85


8BB12
D9HXT
4G85


8BB12
D9HXT
4G85

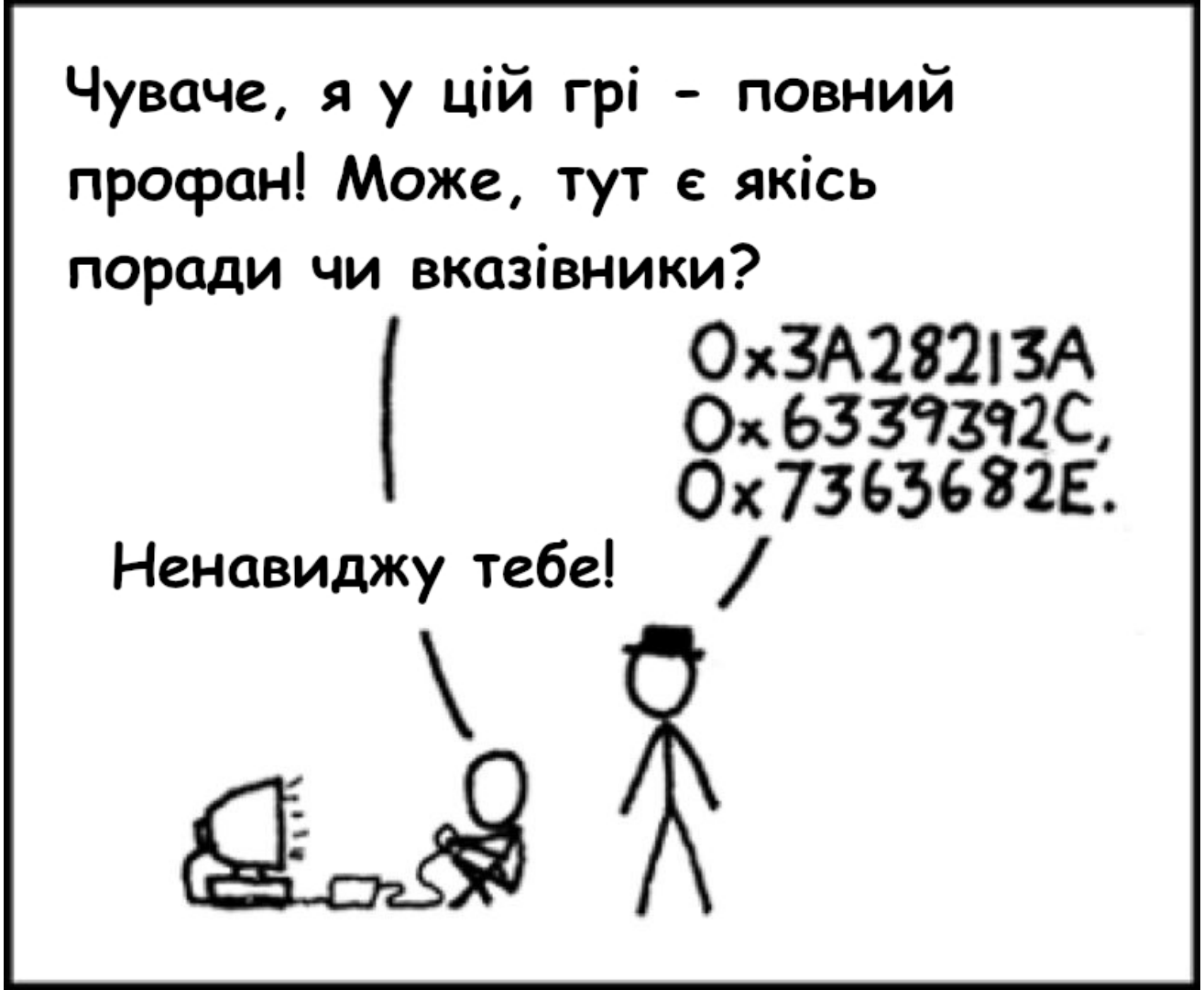
255 216 255

255	216	255
11111111	11011000	11111111

255	216	255
1111 1111	1101 1000	1111 1111

	255		216		255
1111	1111	1101	1000	1111	1111
f	f	d	8	f	f

0xff 0xd8 0xff


Чуваче, я у цій грі - повний профан! Може, тут є якісь поради чи вказівники?
0x3A28213A
0x6339392C,
0x7363682E.
Ненавиджу тебе!

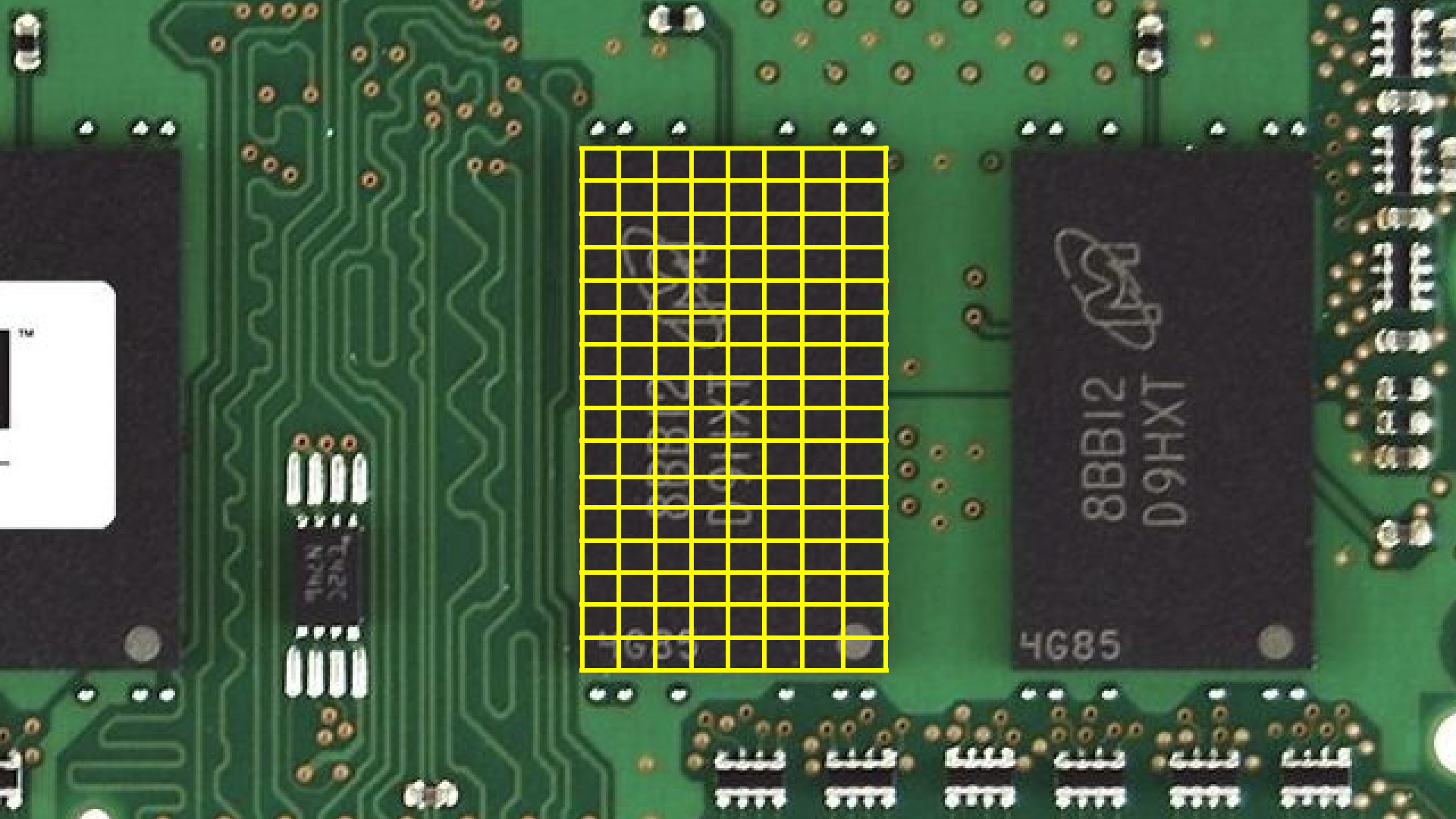
```
void swap(int a, int b)
{



}
```

```
void swap(int a, int b)
{
    int tmp = a;
    a = b;
    b = tmp;
}
```

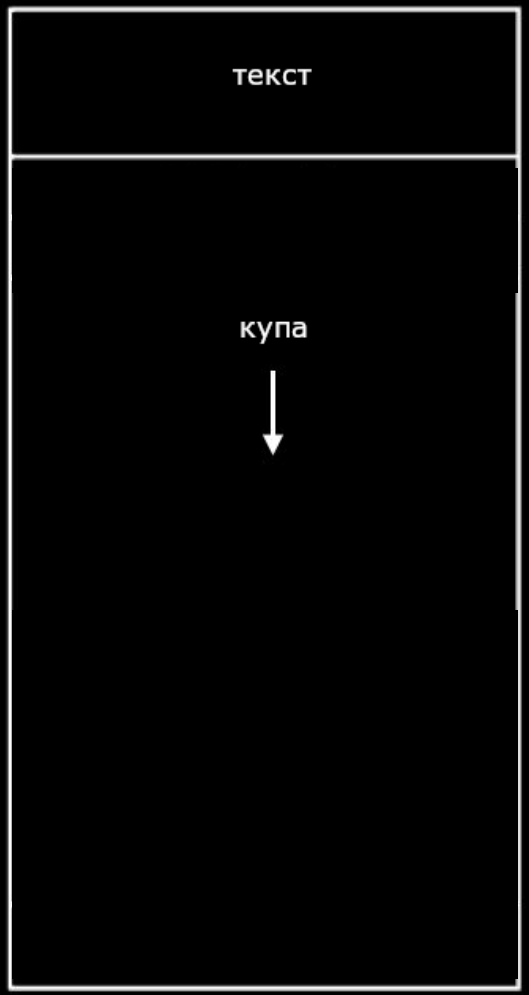
```
void swap(int a, int b)
{
    int tmp = a;
    a = b;
    b = tmp;
}
```

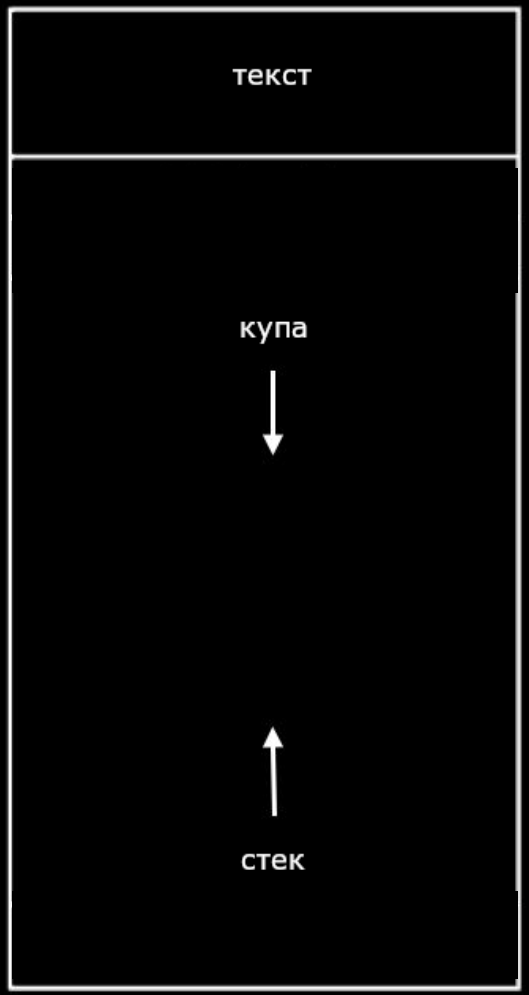
```
void swap(int a, int b)
{
    int tmp = a;
    a = b;
    b = tmp;
}
```

```
void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

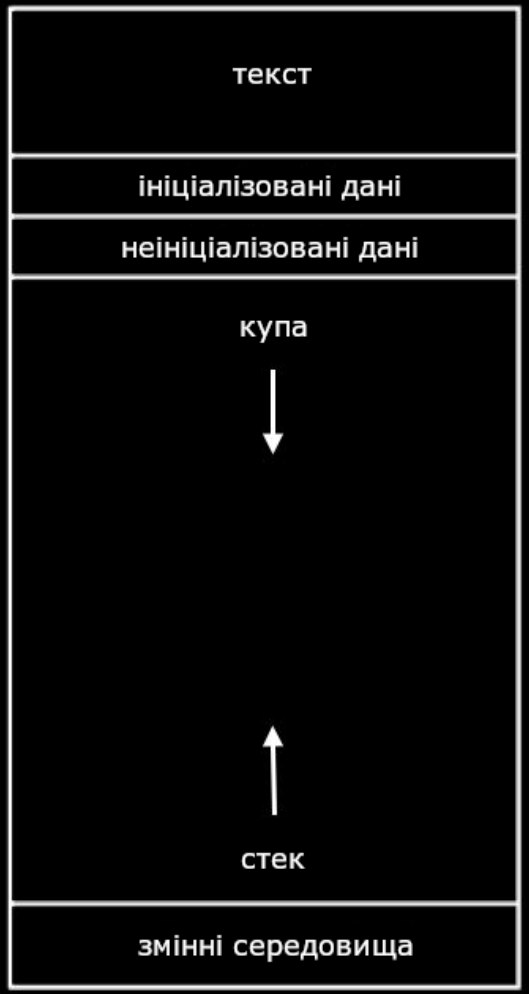

8BB12
D9HXT
4G85

текст


текст
купа


текст
купа
стек

текст
ініціалізовані дані
неініціалізовані дані
купа
стек


текст
ініціалізовані дані
неініціалізовані дані
купа
стек
змінні середовища

```
int main(void)
{
    int *x;
    int *y;

    x = malloc(sizeof(int));

    *x = 42;
    *y = 13;

    y = x;

    *y = 13;
}
```

```c
int main(void)
{
    int *x;
    int *y;

    x = malloc(sizeof(int));

    *x = 42;
    *y = 13;

    y = x;

    *y = 13;
}
```

```c
int main(void)
{
    int *x;
    int *y;

    x = malloc(sizeof(int));

    *x = 42;
    *y = 13;

    y = x;

    *y = 13;
}
```

```c
int main(void)
{
    int *x;
    int *y;

    x = malloc(sizeof(int));

    *x = 42;
    *y = 13;

    y = x;

    *y = 13;
}
```

```c
int main(void)
{
    int *x;
    int *y;

    x = malloc(sizeof(int));

    *x = 42;
    *y = 13;

    y = x;

    *y = 13;
}
```

```c
int main(void)
{
    int *x;
    int *y;

    x = malloc(sizeof(int));

    *x = 42;
    *y = 13;

    y = x;

    *y = 13;
}
```

```c
int main(void)
{
    int *x;
    int *y;

    x = malloc(sizeof(int));

    *x = 42;
    *y = 13;

    y = x;

    *y = 13;
}
```

Pointer Fun with Binky

by Nick Parlante

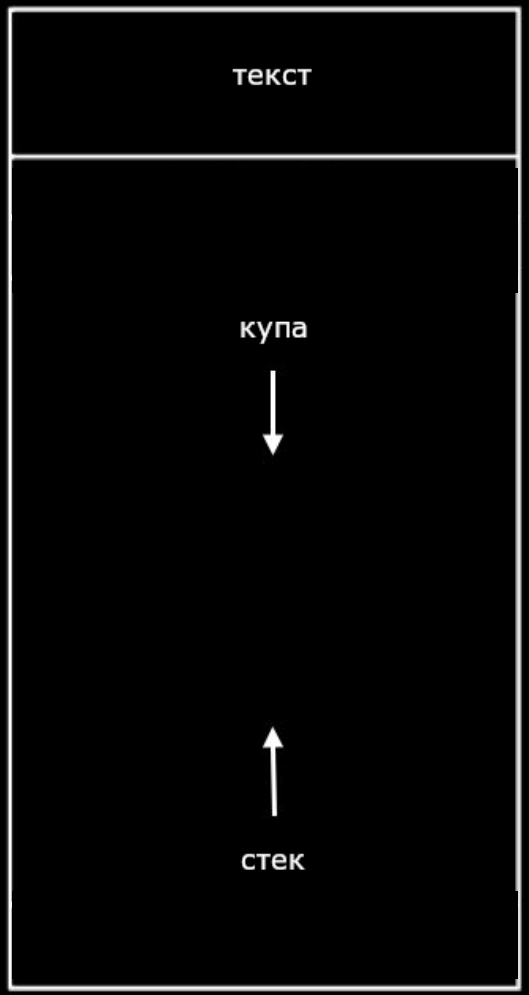
This is document 104 in the Stanford CS Education Library — please see cslibrary.stanford.edu for this video, its associated documents, and other free educational materials.

Copyright © 1999 Nick Parlante. See copyright panel for redistribution terms.

Carpe Post Meridiem!

valgrind

stackoverflow


текст
купа
стек

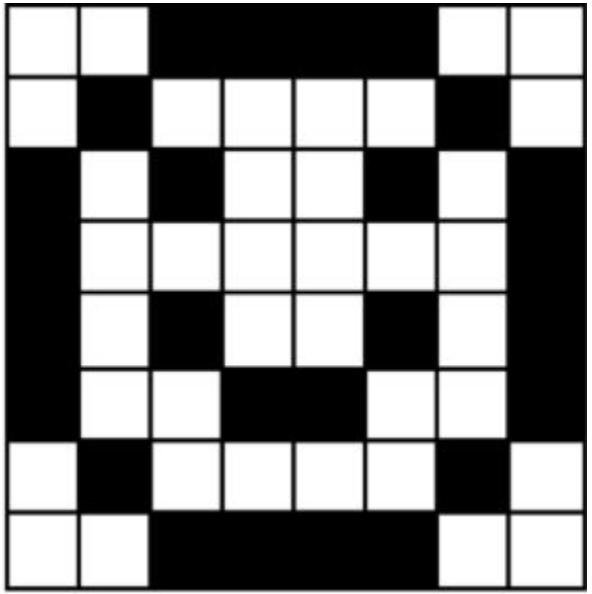
переповнення стеку

переповнення купи

переповнення буферу

struct

```
11000011
10111101
01011010
01111110
01011010
01100110
10111101
11000011
```



offset	type	name	
0	WORD	bfType	BITMAPFILEHEADER
2	DWORD	bfSize	
6	WORD	bfReserved1	
8	WORD	bfReserved2	
10	DWORD	bfOffBits	
14	DWORD	biSize	BITMAPINFOHEADER
18	LONG	biWidth	
22	LONG	biHeight	
26	WORD	biPlanes	
28	WORD	biBitCount	
30	DWORD	biCompression	
34	DWORD	biSizeImage	
38	LONG	biXPelsPerMeter	
42	LONG	biYPelsPerMeter	
46	DWORD	biClrUsed	
50	DWORD	biClrImportant	
54	BYTE	rgbtBlue	RGBTRIPLE
55	BYTE	rgbtGreen	
56	BYTE	rgbtRed	
57	BYTE	rgbtBlue	RGBTRIPLE
58	BYTE	rgbtGreen	
59	BYTE	rgbtRed	
...			
243	BYTE	rgbtBlue	RGBTRIPLE
244	BYTE	rgbtGreen	
245	BYTE	rgbtRed	

Це CS50